

ReaLM: Reliable and Efficient Large Language Model Inference with Statistical Algorithm-Based Fault Tolerance

Tong Xie²¹, Jiawang Zhao¹, Zishen Wan⁵, Zuodong Zhang²⁴, Yuan Wang²³, Runsheng Wang²³⁴, Ru Huang²³⁴, and Meng Li^{123*}

¹*Institute for Artificial Intelligence & ²School of Integrated Circuits, Peking University, Beijing, China*

³*Beijing Advanced Innovation Center for Integrated Circuits, Beijing, China*

⁴*Institute of Electronic Design Automation, Peking University, Wuxi, China*

⁵*Georgia Institute of Technology, Atlanta, GA, USA*

Abstract—The demand for efficient large language model (LLM) inference has propelled the development of dedicated accelerators. As accelerators are vulnerable to hardware faults due to aging, variation, etc, existing accelerator designs often reserve a large voltage margin or leverage algorithm-based fault tolerance (ABFT) techniques to ensure LLM inference correctness. However, previous methods often overlook the inherent fault tolerance of LLMs, leading to high computation and energy overhead. To enable reliable yet efficient LLM inference, in this paper, we propose a novel algorithm/circuit co-design framework, dubbed ReaLM. For the first time, we systematically characterize the fault tolerance of LLMs by performing a large-scale error injection study of representative LLMs and natural language understanding tasks. Then, we propose a statistical ABFT algorithm that fully leverages the error robustness to minimize error recovery as much as possible. We also customize the error detection circuits to enable a low-cost online collection of error statistics. Extensive experiments show that with only 1.42% circuit area and 1.79% power overhead, our ReaLM can reduce perplexity degradation from 18.54 to 0.29. Compared to existing methods, ReaLM consistently reduces recovery costs across different operating voltages and improves energy efficiency by up to 35.83% without compromising LLM performance. Our error injection code is available at https://github.com/PKU-SEC-Lab/ReaLM_DAC25/.

Index Terms—Large language model, algorithm-based fault tolerance, algorithm/circuit co-design

I. INTRODUCTION

Large language models (LLMs) have demonstrated remarkable performance across various tasks such as natural language processing [1], text generation [2], etc. With trillions of operations, LLMs are widely deployed on customized accelerators such as TPU-like systolic arrays (SAs) [3], [4] to meet their enormous computational demands. However, due to aging, process variation, noise, etc [5]–[8], reliability issues, e.g., timing violations, may emerge and make it challenging to ensure the computational correctness on these accelerators. Therefore, how to enable reliable yet efficient LLM acceleration becomes an important question and attracts increasing attention.

Although a large voltage margin can be reserved to prevent timing errors, it is usually not preferred due to the drastic increase in hardware costs. Meanwhile, directly reducing the voltage margin may raise the computation bit error rate (BER) and result in severe performance degradation, as shown in Fig. 1(a). To enable reliable yet efficient computation, various fault mitigation techniques from the circuit to the algorithm level have been proposed but often remain costly in the context of LLM inference. At the circuit level, techniques such as redundancy [9], [10], and Razor flip-flops (FFs) [11]–[14] are developed for traditional logic circuits but usually lack scalability for deep learning accelerators. At the algorithm level, fault-aware fine-tuning has been suggested for conventional convolutional neural networks (CNNs) [15]–[17]. However, the associated costs

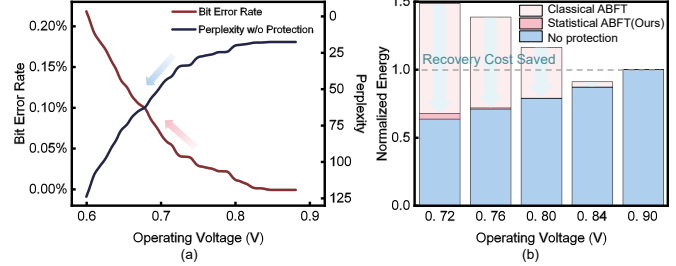


Fig. 1. (a) Lower operating voltages increase BER, leading to significant perplexity degradation without protection. (b) Leveraging model resilience can reduce recovery costs. BERs are synthesized on an SA with commercial 14nm PDK (nominal voltage: 0.9V), aligning with prior studies [11], [22], [23]. Perplexity is evaluated using OPT-1.3B on WikiText-2 dataset.

for LLM fine-tuning are prohibitive. Algorithm-based fault tolerance (ABFT) [18], an algorithm and circuit co-design technique designed for general matrix-matrix multiplication (GEMM), emerges as a cost-effective solution for error detection. ABFT detects errors in the computation results and triggers error recovery mechanisms, e.g., overvolting in dynamic voltage frequency scaling (DVFS) [19]–[21], recomputation, leading to much better efficiency.

However, as shown in Fig. 1(b), we observe that ABFT still faces efficiency challenges due to the high recovery costs. This is because classical ABFT triggers recovery for each detected computation error, which may not be necessary. As in Fig. 1(a), LLM performance does not degrade immediately at low BER, indicating inherited error resilience. By leveraging the error resilience, unnecessary recovery can be minimized to boost the system’s energy efficiency. Nevertheless, the error resilience of LLMs is still not yet well understood. While past works have studied CNNs extensively [24]–[27], the complex architecture, diversified computation patterns, and unique quantization strategies of LLMs can make it significantly different.

Therefore, to fully exploit this inherent error resilience, we propose ReaLM, an algorithm-circuit co-design framework that features reliable and efficient LLM inference. To the best of our knowledge, this is the first work that systematically and comprehensively characterizes the error resilience of LLMs. Our investigation reveals significant variations in error resilience across LLM network components due to the normalization operations. We also identify a trade-off between error magnitude and frequency in their impact on model performance. Building on these characterization insights, ReaLM further proposes a statistical ABFT approach to enable reliable and efficient LLM inference. ReaLM leverages the inherent resilience variations in LLMs to adaptively protect LLM network components in a cost-effective manner. We summarize our contributions as follows:

- We perform a large-scale error injection study on representative LLMs and systematically characterize how various hardware faults affect LLM performance. Our characterization reveals significant inherent resilience variations across LLM network components and identifies a trade-off between error magnitude

*Corresponding author: meng.li@pku.edu.cn

This work was supported in part by National Natural Science Foundation of China under Grant 62495102 and Grant 92464104, in part by Beijing Municipal Science and Technology Program under Grant Z241100004224015, and in part by 111 Project under Grant B18001.

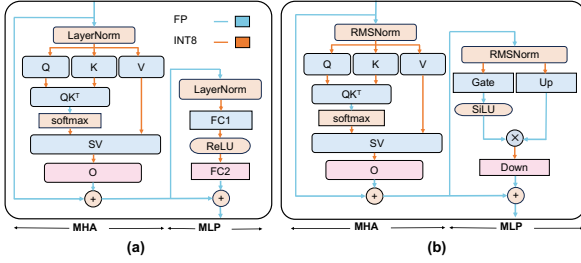


Fig. 2. Transformer blocks of (a) OPT and (b) LLaMA.

and frequency.

- Based on the characterization insights, we propose a statistical ABFT strategy featuring an adaptive error correction *algorithm* and a customized low-cost error detection *circuit*. The proposed strategy integrates seamlessly with SA, minimizing recovery costs while maintaining performance. It further enables energy savings by leveraging LLM's inherent resilience characteristics.
- Experimental results demonstrate that the proposed ReaLM significantly enhances error tolerance. ReaLM reduces perplexity degradation from 18.54 to 0.29 with only 1.42% circuit area and 1.79% power overhead, preserving LLM performance while significantly reducing recovery costs and delivering up to 35.83% energy savings compared to state-of-the-art methods.

II. BACKGROUND

A. Quantized Inference of Generative LLMs

A typical LLM architecture primarily consists of stacked Transformer blocks, each containing several GEMM operations and various nonlinear functions. Fig. 2(a) and (b) illustrate the architectures of two popular Transformer variants, namely the OPT block [28] and the LLaMA block [29], respectively. Both variants consist of various **network components**, as labeled in Fig. 2, which will be referred to as Q, Down, and so on throughout this paper. Existing works [30]–[32] have enabled low bit-width integer quantization of these GEMM operations, while nonlinear functions are retained in floating-point (FP) for accuracy. The generative inference process of an LLM is divided into two stages: the prefill stage takes a prompt sequence as input, and the decode stage produce a single output token each time.

B. Fault-Tolerant Deep Learning

Error sources. Hardware accelerators are susceptible to various permanent and transient faults. Permanent faults often result from fabrication defects, while transient faults primarily arise from timing errors [7], [35]. As CMOS technology scales down to the nanoscale, effects such as aging, variation, and noise become increasingly pronounced [5]–[8], while voltage underscaling further reduces the design margin intended to mitigate these issues. Together, these factors lead to unintended timing delays, ultimately causing timing errors, as shown in Fig. 1(a).

Model resilience. To effectively characterize the inherent resilience of neural networks, many works [22], [36]–[39] have employed a single-bit flip error model. [25] analyzed fault propagation in networks, and [26] assessed how quantization improves fault tolerance. [27] identified vulnerability differences of certain feature maps and suggested selective protection. [23], [40] further extended resilience studies to reinforcement learning applications. While [41] explored error resilience in LLMs, they did not quantify the relationship between errors and model performance. Considering the intricate model architecture, diversified network components, and unique quantization strategies, the error resilience of LLMs can be significantly different from traditional CNN models.

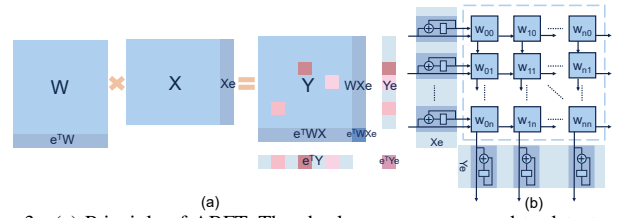


Fig. 3. (a) Principle of ABFT. The checksums are compared to detect errors and capture error statistics. (b) Implementation of ABFT on SA [44].

Error mitigation techniques. Error correction code (ECC) [42] can effectively address hardware faults in memory or on-chip buffers. Mitigating transient faults in computational datapaths is more challenging [38]–[40], [43], where various levels of protection can be leveraged, as summarized in Tab. I. However, these methods incur significant hardware or runtime overhead, particularly for LLMs. For example, redundancy methods [9], [10] such as double-modular redundancy (DMR), require multiple copies of computations to be performed concurrently. Circuit-level techniques [11]–[14] integrate shadow FFs to detect timing errors but lack scalability in modern accelerators. Algorithmic-level approaches [15]–[17] incorporate faults in fine-tuning to enhance error resilience. However, retraining LLMs is prohibitively computation-intensive.

C. Algorithm-Based Fault Tolerance (ABFT)

ABFT leverages checksums to enhance reliability in GEMM [18], as depicted in Fig. 3(a). In classical ABFT, matrices W and X are augmented with a checksum row $e^T W$ and column $X e$ for error detection, where e is a unit column vector. Recent studies [33], [34], [44]–[46] also propose various lightweight detection algorithms which only compute one-side checksums, i.e., $e^T W X$ or $W X e$, or matrix sum deviation (MSD), i.e., $e^T W X e$, to lower ABFT hardware cost, as shown in Fig. 3(a). We also show an example of integrating ABFT with the SA following [44] in Fig. 3(b). While ABFT is suitable for LLMs that depend extensively on GEMM operations, it relies on precise checksum-based comparisons and overlooks the inherent resilience of neural networks. Recent work ApproxABFT [45] uses MSD to assess the error significance and allow small computational errors. However, this method ignores the impact of error frequency on the model accuracy and can lead to unnecessary recomputations.

Therefore, to address these challenges, we introduce ReaLM, a circuit-algorithm co-design framework designed to enhance the reliability and efficiency of LLM inference. We first inject computational errors into LLMs to study their resilience characteristics. Based on this characterization, we determine an adaptive error detection strategy to avoid unnecessary error recovery. Guided by this strategy, we customize a low-cost circuit to capture the computational error statistics.

III. ERROR INJECTION FRAMEWORK

To study the resilience of LLMs, we performed statistical error injection experiments, the most widely used approach for assessing the effects of hardware faults on various models.

A. Error Model

Given that (i) permanent errors are relatively straightforward to detect [47] and do not require mitigation during runtime, and (ii) hardware faults in memory can be effectively addressed using ECC [27], [38]–[40], [43], in this work, we focus on transient computational errors occurring during inference and assume that data is correctly fetched from memory elements. To model hardware faults, we adopt the commonly used random bit-flip model [22], [36]–[39] as an abstraction. Specifically, timing errors are modeled by injecting

TABLE I
COMPARISON OF REPRESENTATIVE FAULT MITIGATION TECHNIQUES.

Method	Level	Detection Capability	Hardware Efficiency	Recovery Efficiency	Recovery Capability	Scalability	Compatibility w/ Accelerators
Redundancy [9], [10]	circuit	high	low	low	high	medium	medium
Razor FFs [11]–[14]	circuit	high	low	medium	low	low	low
Fault-aware Fine-tuning [15]–[17]	algorithm	-	-	prohibited	-	low	-
Classical ABFT [18], [33], [34]	circuit-algorithm	high	medium	low	high	high	high
Ours	circuit-algorithm	high	high	high	high	high	high

errors to higher bits, since they often affect more significant bits [7], [22], [46]. The severity of transient faults can be controlled by varying bit error rates.

B. Error Injection Method

We conduct our error injection study on two representative LLMs: OPT-1.3B [28] and LLaMA-2-7B [29]. To simulate computational errors, we developed a dynamic error injection simulation framework using PyTorch, wherein bit flips are emulated as tensor operations during runtime. Following [30], the inputs of GEMM are quantized to INT8, while the results are in INT32 format. Errors are then injected into the INT32 results of GEMM to realistically emulate compute errors in accelerators, which aligns with previous works [36]–[39].

Furthermore, to discern the different impacts of individual large errors versus multiple smaller errors that yield the identical MSD, we inject identical errors with defined error magnitude (mag) and error frequency ($freq$), adhering to the relationship $MSD = freq \times mag$. It is important to note that while this method diverges from the error models discussed in Sec. III-A, its effectiveness in assessing the resilience of LLMs is preserved.

C. Performance Benchmarks

To assess the effects of error injection, we also construct a comprehensive benchmark framework. Consistent with previous studies [30]–[32], we evaluate model performance using the LAMBADA commonsense QA benchmark [48] and the WikiText-2 language modeling task [49]. Performance on LAMBADA is measured by accuracy ($\uparrow$), while WikiText-2 is evaluated using perplexity ($\downarrow$) [50]. We further capture the autoregressive generation capabilities of LLMs using the X-Sum [51] text summarization task, measured by ROUGE-1 [52] scores ($\uparrow$), and the GSM8K [53] arithmetic reasoning task, evaluated by accuracy ($\uparrow$). Our framework can be extensible to other benchmarks, like Hellaswag [54] (measured by accuracy ($\uparrow$)), which we examine in the evaluation section.

IV. LLM RESILIENCE CHARACTERIZATION

In this section, we present our LLM resilience characterization in detail, setting out to answer the following research questions:

- **Q1.1:** How does resilience vary across different layers of LLMs? (Sec. IV-A1)
- **Q1.2:** What is the bit-wise resilience of LLMs? (Sec. IV-A2)
- **Q1.3:** How does resilience vary among different components within LLMs during the prefill stage? (Sec. IV-A3)
- **Q1.4:** What is the correlation between error magnitude and frequency in impacting LLM performance? (Sec. IV-A4)
- **Q2.1:** How does the resilience of LLMs compare between the prefill stage and the decode stage? (Sec. IV-B1)
- **Q2.2:** How does resilience vary among different computational components within LLMs during the decode stage? (Sec. IV-B2)

To control for irrelevant variables, we implement the following protocols across research questions. (1) For Q1.4, we exclusively

use OPT-1.3B model for fast assessment, whereas for Q2.1 and Q2.2, we employ only LLaMA-2-7B model to ensure reasonable performance. For all other questions, both models are utilized. (2) Error injections are applied to all layers for all questions, except for Q1.1. (3) In Q1.1, Q1.3, Q2.1 and Q2.2, we specifically target the 30th bit for flipping. (4) For Q1.1 and Q2.1, errors are injected across all network components within a single Transformer block simultaneously.

A. Resilience Study on Prefill Stage

In this subsection, we will directly give the conclusion of Q1.1 and Q1.2 since they are relatively straightforward, and we focus on discussing Q1.3 and Q1.4.

1) *Layer-wise resilience:* As shown in Fig. 4(a)(b), **while the layers exhibit comparable resilience behaviors, the earlier layers are more vulnerable to errors.**

2) *Bit-wise resilience:* By comparing error impacts in network components with FP outputs (O, Fig. 4(d)) and re-quantized INT8 outputs (K, Fig. 4(c)), we conclude that **while errors at lower bits have a negligible impact on model performance, errors at higher bits can reach a saturation point due to re-quantization.**

3) *Resilience of different network components:* As depicted in Fig. 4 (e)(f), our results reveal substantial differences in resilience behavior across the network components. Specifically, the O component in both models, FC2 in OPT-1.3B, and Down in LLaMA-2-7B exhibit significantly higher sensitivity compared to other components. For ease of discussion, these components are referred to as **sensitive components**, while the others are categorized as **resilient components**.

Discussion: As demonstrated in Fig. 2, the sensitive components in both OPT-1.3B and LLaMA-2-7B are followed by normalization layers—LayerNorm and RMSNorm, respectively, unlike their resilient counterparts. We demonstrate in Fig. 5 that upon introducing a single error into the pre-normalization hidden states, a dramatic alteration in the subsequent elements is observed. This substantial variation is primarily due to the calculations of mean (μ) and standard deviation (σ) during normalization. In LLMs, hidden states inherently consist of a few outliers [30]–[32], while the majority of elements remain close to zero. These outliers predominantly influence the computation of μ and σ , shaping the overall statistics of the hidden states. Therefore, a larger error can manifest as an outlier and drastically skew these parameters, markedly altering the normalization outcome. In contrast, injecting a larger error into the resilient components affects only a localized set of elements. Thus, we conclude that **the network components followed by normalization exhibit much worse resilience than other components due to normalization.** Specifically, for $\mathbb{QK}^T$ components followed by softmax, changes remain confined because softmax outputs between 0 and 1.

4) *Tradeoff between error magnitude and frequency:* While MSD can serve as a metric to depict the error severity, it cannot differentiate

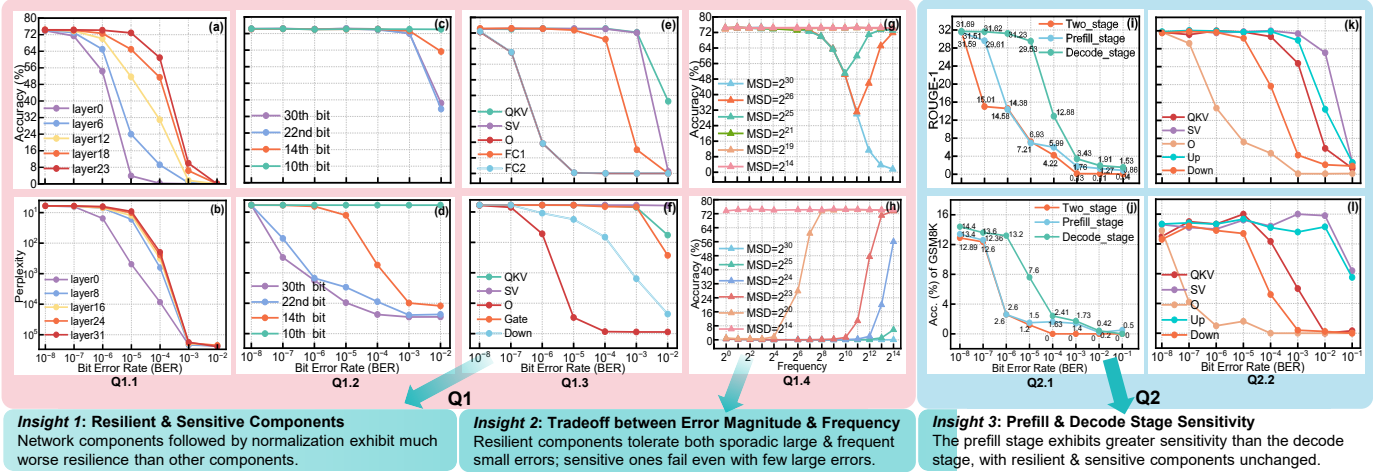


Fig. 4. **Q1.1:** (a)(b) Layer-wise resilience of different LLMs on different tasks. **Q1.2:** Bit-wise error resilience. (c) Error injection on K. (d) Error injection on O. **Q1.3:** (e)(f) Sensitivity to errors in different LLM components. **Q1.4:** Relationship between error frequency and magnitude. (g) Resilient components like K. (h) Sensitive components like O. Given MSD, the error magnitude decreases as the error frequency increases. **Q2.1:** (i)(j) Comparison between the prefill stage and decode stage. **Q2.2:** (k)(l) Impact of error injection across network components: O. and Down remain highly sensitive. (a)(c)(e)(g)(h) are evaluated with OPT-1.3B on LMDABA; (b)(d)(f) with LLaMA-2-7B on WikiText-2; (i)(k) with LLaMA-2-7B on X-Sum; (j)(l) with LLaMA-2-7B on GSM8K.

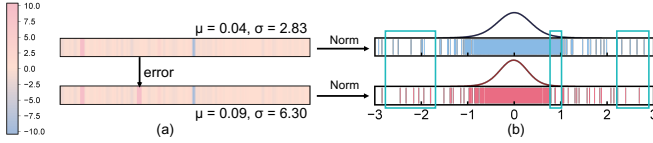


Fig. 5. (a) Data distribution of the pre-norm layer in LLMs, where outliers dominate μ and σ . Injecting larger errors can cause significant skew. (b) Data distribution after normalization is largely affected by the injected error.

between the impacts of a single large error and multiple smaller errors, despite them potentially yielding identical MSD values and having significantly different effects on model performance. Consequently, this section introduces additional metrics—error magnitude (*mag*) and error frequency (*freq*)—to more accurately represent error statistics. We define *mag* as $MSD/freq$ in Fig. 4(g)(h), indicating that an increase in *freq* results in a decrease in *mag*. Our analysis reveals that resilient components exhibit non-monotonic resilience behavior: both sporadic large errors and frequent smaller errors minimally impact model performance. Conversely, a moderate frequency of medium-sized errors can severely impair performance. However, for sensitive components, acceptable performance is maintained only when numerous smaller errors are introduced.

Discussion. The non-monotonic resilience behavior observed in resilient components illustrates a trade-off between error magnitude and frequency. **Resilient components are tolerant to both sporadic large errors and frequent smaller ones, whereas sensitive components falter even with few large errors.** This finding aligns with earlier observations from Sec. IV-A3. Fig. 6 further delineates a critical region where significant performance degradation occurs.

B. Resilience Study on Decode Stage

LLMs are primarily different from CNNs due to their autoregressive decode stage. In this section, we explore the resilience characterization of the decode stage and differentiate it from the prefill stage.

1) *Difference in prefill stage and decode stage:* To differentiate resilience between the prefill and decode stages, we inject errors exclusively into the prefill stage, the decode stage, and both stages, respectively. As illustrated in Fig. 4(i)(j), our analysis indicates that performance degradation is consistently more severe during the prefill stage than during the decode stage across both tasks.

Discussion. The prefill stage exhibits greater sensitivity to errors than the decode stage. The reason lies in the KV-cache mechanism. During the decode stage, the newly generated query vector interacts with the KV-cache, which is primarily composed during the prefill stage. Consequently, errors during the prefill stage compromise most of the operations involved in generating the next token. In contrast, errors during the decode stage have a lesser impact, as the largely unaffected KV cache drives most of computations for the next token generation.

2) *Resilience across compute units:* Similar to the experiments described in Sec. IV-A3, we inject errors into various LLM network components and present the outcomes in Fig. 4(k)(l). Our findings confirm that **the sensitive components, specifically O and Down, continue to be highly vulnerable**, echoing observations from Sec. IV-A3.

V. STATISTICAL ALGORITHM-BASED FAULT TOLERANCE

Building on the insights from Sec. IV, this section introduces our statistical ABFT framework. We integrate it with SAs, focusing on weight stationary (WS) and output stationary (OS) dataflows due to page limitations, though it is also compatible with input stationary (IS) dataflow.

A. Statistical-Based Error Detection Strategy

ReaLM utilizes MSD, error frequency (*freq*), and error magnitude (*mag*) as key statistics to characterize the error distributions. As shown in Fig. 6, an *acceptable* performance threshold can be empirically established to define the critical region where computational errors significantly impact LLM performance. ReaLM simply ignores errors outside the region.

In terms of the critical region, we find that for resilient components, the boundary of the critical error region is defined by a horizontal line and an inclined line with a slope greater than 1. The horizontal line suggests that resilient components are tolerable to errors below a frequency threshold θ_{freq} , i.e., the vertical coordinate of this line, regardless of their magnitude. For sensitive components, the critical region is bounded by the inclined line and horizontal axis. Note that the slope $a > 1$, the intercept $-b$ of the inclined line, and frequency threshold θ_{freq} are all fitted parameters.

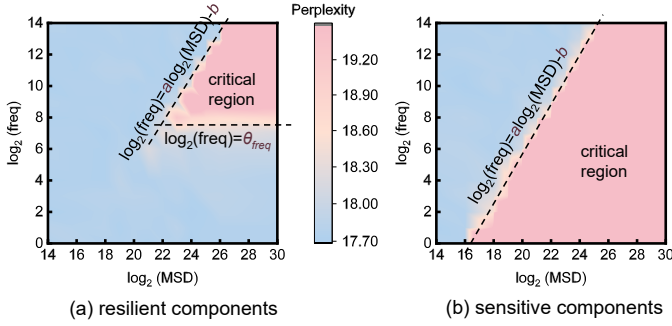


Fig. 6. Our statistical ABFT strategy only corrects errors falling inside the critical region.

We derive several key conclusions to guide our strategy: 1) the horizontal bound indicates that errors exceeding a certain MSD or magnitude all lead to unacceptable performance degradation at θ_{freq} . Therefore, errors larger than the threshold can be counted without distinction; 2) Errors positioned above the inclined line are deemed acceptable. This boundary delineates a clear threshold θ_{mag} beneath which errors, regardless of their frequency, do not significantly degrade the model performance; 3) Assuming that errors are equal, we can derive the upper bound of $\theta_{mag} = b - (a - 1) \log_2 MSD$. Errors smaller than θ_{mag} have negligible impact on model performance.

Based on these insights, we assess the influence of errors with error statistics in real-time. Our methodology ignores errors smaller than θ_{mag} due to their minimal impact on performance. Instead, we only count significant errors to calculate the effective error frequency $freq_{eff} = \text{countif}(mag > \theta_{mag})$. We then compare $freq_{eff}$ to the threshold θ_{freq} ; if $freq_{eff} > \theta_{freq}$, the recovery is triggered. Otherwise, the GEMM operation is considered safe.

B. Architecture Design of Statistical ABFT

We integrate our statistical ABFT into SA architecture and support both WS and OS dataflow (Fig. 7). We only need to add an extra “statistical unit”, as shown in Fig. 7(c), to capture the error statistics and determine the need for recovery, which incurs a very small area overhead. For ease of explanation, we consider the example of computing a GEMM of matrices $W, X \in \mathbb{R}^{n \times n}$ on an $n \times n$ SA, resulting in $Y \in \mathbb{R}^{n \times n}$.

In the WS configuration of the SA (Fig. 7(a)), weights are stored within the processing elements (PEs), and inputs are streamed horizontally from left to right. The array computes partial sums that move vertically downward. A column of PEs is added on the right, storing $e^T W$ for computing $e^T W X$. Meanwhile, a row of adders at the bottom accumulates $e^T Y$. This design minimizes the need for higher bit-width multipliers, as seen in [44], with only minimal area cost. The latency overhead of computing $e^T W X$ and $e^T Y$ is also negligible.

Similarly, for the OS dataflow (Fig. 7(b)), weights traverse horizontally from left to right, while inputs flow vertically from top to bottom. Outputs are generated directly at the intersection of weights and inputs within the PEs. We follow previous works [46] for our implementation. An additional column of adders is integrated on the left side of the array, dedicated to computing $e^T W$. This checksum is then routed to an extra row of PEs positioned at the bottom and propagated rightward to calculate the product $e^T W X$. The outputs y are vertically accumulated across the PEs to derive the output checksum $e^T Y$. This configuration only requires the additional row of PEs to support higher-bit multiplications.

The proposed statistical unit comprises a subtractor, an accumulator, a Log2LinearFunction unit, and a sequence of n buffers

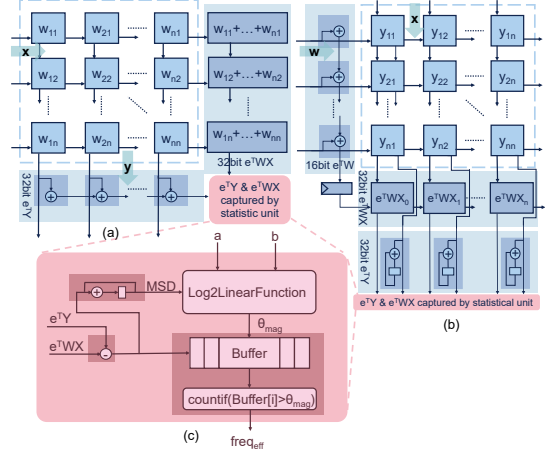


Fig. 7. Architecture design of statistical ABFT on SA: (a) ABFT implementation for WS dataflow; (b) ABFT implementation for OS dataflow; and (c) customized statistical units.

equipped with a comparator-based “countif” unit. The Log2LinearFunction unit is designed to calculate $\theta_{mag} = b - (a - 1) \log_2 MSD$. Throughout the computation process, checksums are continuously fed into the statistical unit. They are first subtracted and the resultant differences are accumulated to compute MSD, which is then stored in the buffers. Once the accumulator completes the MSD computation, the data is forwarded to the Log2LinearFunction unit, where θ_{mag} is computed. Ultimately, the “countif” unit evaluates all buffer elements in parallel and counts the frequency of errors exceeding θ_{mag} to determine $freq_{eff}$.

In the decode stage, general matrix-vector multiplication (GEMV) dominates the computation. While batched GEMVs can also be treated as GEMMs, to which our method applies, GEMVs during non-batched inference are typically executed in specialized vector processing units, which fall outside the primary focus of SAs and ABFT techniques discussed in this paper.

Therefore, based on the insights in Fig. 4, the proposed statistical ABFT can enhance LLM resilience by adaptively leveraging captured error statistics. This, in turn, unlocks significant energy-saving opportunities, such as voltage underscaling, even in the presence of errors (Fig. 1), enabling both reliable and efficient LLM inference.

VI. EVALUATION

A. Experiment Setup

We first use the lightweight OPT-1.3B measuring perplexity on WikiText-2 for rapid evaluation. To demonstrate generality, we also extend the evaluation to the recently released LLaMA-3-8B model and a different task, i.e., HellaSwag [54]. Timing errors are simulated by randomly flipping bits in the accumulation results (represented as 32-bit integers) following [7], [22]. The relation between the BER and the operating voltage follows Fig. 1. To determine the parameters for the proposed statistical ABFT, i.e., a, b, θ_{freq} , we inject errors into LLMs for performance evaluation. We empirically set the parameters to allow a 0.3 perplexity increase and a 0.5% accuracy decrease as the acceptable performance degradation.

The RTL for both our proposed and existing ABFT designs is implemented and integrated into SAs with 256×256 PEs supporting both WS and OS dataflows. These designs are synthesized using Synopsys Design Compiler with commercial 14nm PDK, where the nominal operating voltage is 0.9V. The clock cycle is set to 500 ps, with a critical path delay of 439 ps, and circuit area and power are obtained from synthesis results. We conduct timing

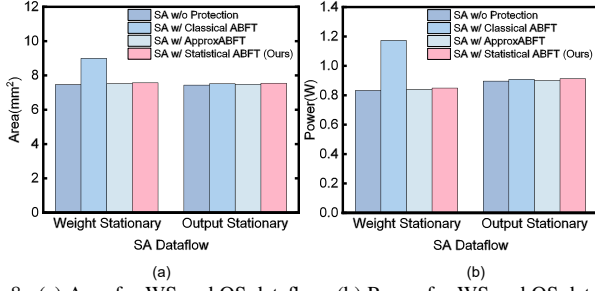


Fig. 8. (a) Area for WS and OS dataflow; (b) Power for WS and OS dataflow.

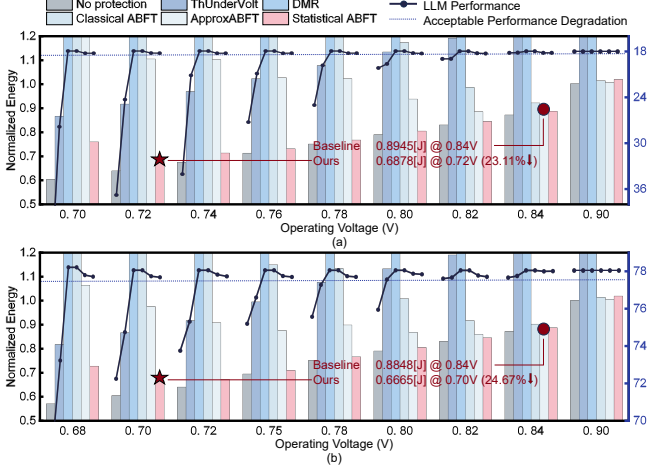


Fig. 9. LLM performance and total energy savings comparison. (a) OPT-1.3B on WikiText-2. (b) LLaMA-3-8B on HellaSwag. Our method preserves competitive performance with minimal protection overhead, achieving maximal energy savings at 0.72V and 0.70V, respectively.

analysis using Synopsys PrimeTime and HSPICE to capture BERs at reduced voltages, considering the toggle rate during real LLM inference, which is in line with previous chip measurement data [11], [22], [23]. Error recovery is assumed to be performed through recomputation at nominal voltage to ensure correctness, and the total energy consumption is evaluated accordingly.

B. Circuit Overhead Comparison

For both WS and OS dataflows, we analyze the circuit overhead in terms of area and power consumption across no protection, classical ABFT [46], ApproxABFT [45], and statistical ABFT, as depicted in Fig. 8. Compared to unprotected SAs, our design introduces minimal overhead, with only 1.43% area and 1.82% power overhead for WS dataflow, and 1.42% area and 1.79% power overhead for OS dataflow.

C. LLM Performance and Energy Savings

We apply our statistical ABFT to network component K in OPT-1.3B and V in LLaMA-3-8B as examples to demonstrate its effectiveness. Fig. 9 compares the total energy overhead and LLM performance for different methods: no protection, ThunderVolt [13], DMR, classical ABFT [46], ApproxABFT [45], and our statistical ABFT, across various operating voltages. Our statistical ABFT maintains comparable LLM performance while significantly reducing the recovery cost and therefore total energy. For K in OPT-1.3B, our approach achieves a sweet spot at 0.72V, reducing perplexity degradation from 18.54 to 0.29 and saving 23.11% energy compared to prior-art methods. Similarly, for V in LLaMA-3-8B, it finds a sweet spot at 0.70V, lowering accuracy degradation from 7.64% to 0.47% and saving 24.67% energy.

TABLE II
ENERGY SAVING ACROSS NETWORK COMPONENTS

OPT-1.3B			LLaMA-3-8B		
Network Component	Optimal Voltage (V)	Energy Saving	Network Component	Optimal Voltage (V)	Energy Saving
Q	0.70	28.68%	Q	0.71	24.26%
K	0.72	23.11%	K	0.72	24.17%
V	0.65	35.83%	V	0.70	24.67%
QK ^T	0.67	33.54%	QK ^T	0.73	19.46%
SV	0.75	17.44%	SV	0.66	34.46%
O	0.82	3.38%	O	0.83	2.40%
FC1	0.75	15.01%	Gate Up	0.78	10.21%
FC2	0.83	3.14%	Down	0.77	16.56%
				0.83	3.12%

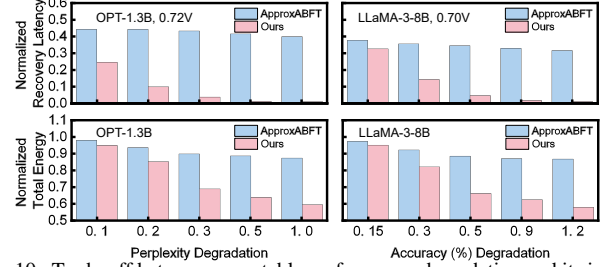


Fig. 10. Trade-off between acceptable performance degradation and its impact on recovery latency and total energy.

D. Energy Savings across LLM Components

We summarize the total energy savings for all network components in Tab. II. As shown, optimal operating voltages (i.e., sweet spots) and resulting benefits vary across components, reflecting the differences in resilience as discussed in Sec. IV. Our method achieves up to 35.83% energy savings on V in OPT-1.3B and 34.46% on SV in LLaMA-3-8B. While resilient components demonstrate benefits, sensitive components (O, FC2, and Down) show more limited savings due to their vulnerability to errors, which reduces the potential to exploit inherent resilience.

E. Tradeoff between Performance and Energy Savings

We explore the tradeoff between acceptable performance degradation and its impact on recovery latency and total energy over baseline [45] for K in OPT-1.3B and V in LLaMA-3-8B, as shown in Fig. 10. The recovery latency is evaluated at 0.72V and 0.70V, respectively, and the total energy is evaluated at the optimal voltages on each performance degradation constraint. This tradeoff demonstrates our design's flexibility in balancing computational efficiency with reliability requirements.

VII. CONCLUSION

In this paper, we introduce ReaLM, an algorithm/circuit co-design framework aimed at enhancing the reliability and efficiency of LLM inference. We conduct a systematic error injection experiment to characterize the resilience of LLMs, highlighting particular vulnerabilities in the normalization operation that significantly affect inference reliability. Building on these insights, we develop a statistical ABFT approach featuring a customized, low-cost error detection circuit and an adaptive error correction strategy, effectively reducing error recovery costs. Our evaluations reveal that ReaLM can significantly enhance LLM performance by reducing perplexity degradation from 18.54 to 0.29 with only 1.42% circuit area overhead and 1.79% power overhead. By fully leveraging the inherent resilience of LLMs, ReaLM can achieve up to 35.83% energy savings compared to existing methods while maintaining competitive model performance.

REFERENCES

- [1] T. B. Brown *et al.*, “Language models are few-shot learners,” in *Proc. NIPS*, pp. 1877–1901, 2020.
- [2] W. X. Zhao *et al.*, “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [3] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proc. ISCA*, pp. 1–12, 2017.
- [4] S. Markidis *et al.*, “Nvidia tensor core programmability, performance & precision,” in *2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*, pp. 522–531, IEEE, 2018.
- [5] H. D. Dixit *et al.*, “Silent data corruptions at scale,” *arXiv preprint arXiv:2102.11245*, 2021.
- [6] I. Moghaddasi *et al.*, “Dependable dnn accelerator for safety-critical systems: A review on the aging perspective,” *IEEE Access*, 2023.
- [7] X. Jiao *et al.*, “Clim: A cross-level workload-aware timing error prediction model for functional units,” *IEEE Transactions on Computers*, vol. 67, no. 6, pp. 771–783, 2017.
- [8] R. Huang *et al.*, “Variability-and reliability-aware design for 16/14nm and beyond technology,” in *Proc. IEDM*, pp. 12–4, IEEE, 2017.
- [9] F. Libano *et al.*, “Selective hardening for neural networks in fpgas,” *IEEE Transactions on Nuclear Science*, vol. 66, no. 1, pp. 216–222, 2018.
- [10] N. Khoshavi *et al.*, “Shieldenn: Online accelerated framework for fault-tolerant deep neural network architectures,” in *Proc. DAC*, pp. 1–6, IEEE, 2020.
- [11] D. Ernst *et al.*, “Razor: A low-power pipeline based on circuit-level timing speculation,” in *Proc. MICRO*, pp. 7–18, IEEE, 2003.
- [12] P. N. Whatmough *et al.*, “Dnn engine: A 28-nm timing-error tolerant sparse deep neural network processor for iot applications,” *IEEE Journal Solid-State Circuits*, vol. 53, no. 9, pp. 2722–2731, 2018.
- [13] J. Zhang *et al.*, “Thundervolt: enabling aggressive voltage underscaling and timing error resilience for energy efficient deep learning accelerators,” in *Proc. DAC*, pp. 1–6, 2018.
- [14] N. D. Gundi *et al.*, “Effort: Enhancing energy efficiency and error resilience of a near-threshold tensor processing unit,” in *Proc. ASPDAC*, pp. 241–246, IEEE, 2020.
- [15] C. Schorn *et al.*, “An efficient bit-flip resilience optimization method for deep neural networks,” in *Proc. DATE*, pp. 1507–1512, IEEE, 2019.
- [16] S. Kim *et al.*, “Matic: Learning around errors for efficient low-voltage neural network accelerators,” in *Proc. DATE*, pp. 1–6, IEEE, 2018.
- [17] Z. He *et al.*, “Noise injection adaption: End-to-end reram crossbar non-ideal effect adaption for neural network mapping,” in *Proc. DAC*, pp. 1–6, 2019.
- [18] K.-H. Huang *et al.*, “Algorithm-based fault tolerance for matrix operations,” *IEEE Transactions on Computers*, vol. 100, no. 6, pp. 518–528, 1984.
- [19] H. Cherupalli *et al.*, “Exploiting dynamic timing slack for energy efficiency in ultra-low-power embedded systems,” *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 671–681, 2016.
- [20] J. Constantin *et al.*, “Exploiting dynamic timing margins in microprocessors for frequency-over-scaling with instruction-based clock adjustment,” in *Proc. DATE*, pp. 381–386, IEEE, 2015.
- [21] T. Jia *et al.*, “19.4 an adaptive clock management scheme exploiting instruction-based dynamic timing slack for a general-purpose graphics processor unit with deep pipeline and out-of-order execution,” in *Proc. ISSCC*, pp. 318–320, IEEE, 2019.
- [22] Z. Zhang *et al.*, “Read: Reliability-enhanced accelerator dataflow optimization using critical input pattern reduction,” in *Proc. ICCAD*, pp. 1–9, IEEE, 2023.
- [23] Z. Wan *et al.*, “Mulberry: Enabling bit-error robustness for energy-efficient multi-agent autonomous systems,” in *Proc. ASPLOS*, pp. 746–762, 2024.
- [24] X. Jiao *et al.*, “An assessment of vulnerability of hardware neural networks to dynamic voltage and temperature variations,” in *Proc. ICCAD*, pp. 945–950, IEEE, 2017.
- [25] G. Li *et al.*, “Understanding error propagation in deep learning neural network (dnn) accelerators and applications,” in *Proc. SC*, pp. 1–12, 2017.
- [26] B. Reagen *et al.*, “Ares: A framework for quantifying the resilience of deep neural networks,” in *Proc. DAC*, pp. 1–6, 2018.
- [27] A. Mahmoud *et al.*, “Optimizing selective protection for cnn resilience,” pp. 127–138, 2021.
- [28] S. Zhang *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [29] H. Touvron *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [30] G. Xiao *et al.*, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *Proc. ICML*, pp. 38087–38099, PMLR, 2023.
- [31] J. Lin *et al.*, “Awq: Activation-aware weight quantization for llm compression and acceleration,” *arXiv preprint arXiv:2306.00978*, 2023.
- [32] Z. Yuan *et al.*, “Rptq: Reorder-based post-training quantization for large language models,” *arXiv preprint arXiv:2304.01089*, 2023.
- [33] F. Libano *et al.*, “Efficient error detection for matrix multiplication with systolic arrays on fpgas,” *IEEE Transactions on Computers*, 2023.
- [34] T. Marty *et al.*, “Safe overclocking for cnn accelerators through algorithm-level error detection,” *IEEE TCAD*, vol. 39, no. 12, pp. 4777–4790, 2020.
- [35] B. Salami *et al.*, “On the resilience of rtl nn accelerators: Fault characterization and mitigation,” in *2018 30th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, pp. 322–329, IEEE, 2018.
- [36] B. Sangchoolie *et al.*, “One bit is (not) enough: An empirical study of the impact of single and multiple bit-flip errors,” in *Proc. DSN*, pp. 97–108, IEEE, 2017.
- [37] Y. He *et al.*, “Fidelity: Efficient resilience analysis framework for deep learning accelerators,” in *Proc. MICRO*, pp. 270–281, IEEE, 2020.
- [38] Y. S. Hsiao *et al.*, “Silent data corruption in robot operating system: A case for end-to-end system-level fault analysis using autonomous uavs,” *IEEE TCAD*, 2023.
- [39] G. Papadimitriou *et al.*, “Demystifying the system vulnerability stack: Transient fault effects across the layers,” in *Proc. ISCA*, pp. 902–915, IEEE, 2021.
- [40] Z. Wan *et al.*, “Analyzing and improving fault tolerance of learning-based navigation systems,” in *Proc. DAC*, pp. 841–846, IEEE, 2021.
- [41] U. K. Agarwal *et al.*, “Resilience assessment of large language models under transient hardware faults,” pp. 659–670, IEEE, 2023.
- [42] R. W. Hamming, “Error detecting and error correcting codes,” *The Bell system technical journal*, vol. 29, no. 2, pp. 147–160, 1950.
- [43] P. H. Hochschild *et al.*, “Cores that don’t count,” in *Proceedings of the Workshop on Hot Topics in Operating Systems*, pp. 9–16, 2021.
- [44] S. Bal *et al.*, “A novel fault-tolerant architecture for tiled matrix multiplication,” in *Proc. DATE*, pp. 1–6, IEEE, 2023.
- [45] X. Xue *et al.*, “Approxabft: Approximate algorithm-based fault tolerance for vision transformers,” *arXiv preprint arXiv:2302.10469*, 2023.
- [46] M. Safarpour *et al.*, “Algorithm level error detection in low voltage systolic array,” *IEEE TCAS II*, vol. 69, no. 2, pp. 569–573, 2021.
- [47] R. Isermann, “Process fault detection based on modeling and estimation methods—a survey,” *automatica*, vol. 20, no. 4, pp. 387–404, 1984.
- [48] D. Paperno *et al.*, “The lambda dataset: Word prediction requiring a broad discourse context,” *arXiv preprint arXiv:1606.06031*, 2016.
- [49] S. Merity *et al.*, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [50] F. Jelinek *et al.*, “Perplexity—a measure of the difficulty of speech recognition tasks,” *The Journal of the Acoustical Society of America*, vol. 62, no. S1, pp. S63–S63, 1977.
- [51] S. Narayan *et al.*, “Don’t give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization,” *arXiv preprint arXiv:1808.08745*, 2018.
- [52] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries,” in *Text summarization branches out*, pp. 74–81, 2004.
- [53] K. Cobbe *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [54] R. Zellers *et al.*, “Hellswag: Can a machine really finish your sentence?,” *arXiv preprint arXiv:1905.07830*, 2019.